

ECE486 Lab 4: Vision-Based Movement

In this lab you will:

- Use vision to restrict the workspace of the robot
- Interact with the environment (in a limited way in lab 4!)
- Generate a robot trajectory between two ArUco markers

Understanding the Problem

As of the last lab you are able to calibrate a transformation between a camera and the robot using ArUco markers. You will now be provided with a printout that consists of several ArUco markers scattered across a page. We will consider this page to be a restricted subspace of the robot's workspace – all tasks in this lab must take place on or above that piece of paper, provided that you placed the paper within the robot's workspace from the previous labs. Consider this to be like a manufacturing task where the robot may be able to move broadly about its workspace, but the actual task to be completed is on a smaller workpiece within that broader workspace. Here is an example of what your printout might look like.

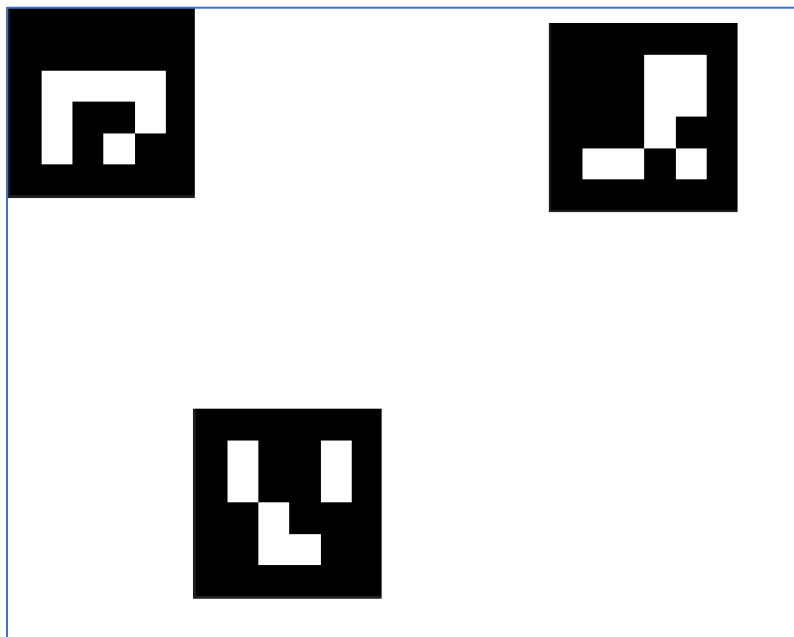


Figure 1: A sample printout

Your printout will have three ArUco markers on it, distributed randomly across the page. The workspace must be restricted as follows:

1. The markers, when transformed into the robot's coordinate frame, will all have x , y , and z coordinates. Restrict the robot's xy motion to the maximum and minimum x and y of these markers, or the robot's original workspace, whichever is more restrictive.
 - For example, if one of the ArUco markers happens to be placed with an x coordinate of 350mm, this is outside of the robot's original workspace

- Think of this as the intersection of a box in the xy plane and the robot's original workspace projected onto the xy plane
 - This workspace restriction depends on the placement of the paper within the original workspace. The markers will always be placed on the page so that all three *can* fit within the original workspace if placed correctly
2. The robot will be fitted with a pen. You must restrict the z coordinates of the robot so that:
 - The pen may be moved freely in the xy plane without making contact with the table
 - i. This may require you to change the robot's upper z limit to be above 0, and it will depend on how your pen is mounted. This is OK.
 - The pen tip may make contact with the paper. The pen holder allows for approximately 1cm of travel using a spring mechanism once contact is made. For safety, keep the allowed travel to no more than 0.5cm
 - You should use Dobot Link to move the robot manually and record its positions for this part. Do not use python code you wrote yourself! That code is what you are restricting with this new workspace, so you can't pre-program a motion if you don't know what those restrictions are.
 3. At certain times it may be useful to move the robot so that it does not obstruct the camera's view. You may move outside of the restricted workspace to do so, provided that the robot does not exceed its limits, *especially* making sure that it does not drive the pen into the table!

When ArUco markers are detected by the camera, what is actually detected is the coordinates of the exact center of the marker. Markers are identified by their ArUco ID. Starter code for finding both ID and coordinates of the center can be found in the `find_aruco.py` script.

Part 1: Calibration Adjustment

Note: Remember to begin by re-calibrating the robot to the camera frame using the scripts from last lab. If you have never used this station before, you will also need to determine the camera matrix.

Begin by placing your printout so that all ArUco markers are within the robot's workspace.

Write a Python script that receives the ID of an ArUco marker. If that marker was detected, the robot should move to the marker's center with its pen off the table, then place a dot using the pen on the marker. The robot should then move to a position chosen so that it does not obstruct the camera's view.

If you compare the location of the dot to the location of the center of the marker using a ruler, you will see that your robot probably did not perfectly hit the center. Your goal is to adjust the robot's position until this error is as close to zero as possible for all three markers. You will likely need to run this experiment several times, collecting error data, and adjust your robot movement script until the error is minimized. Be exceptionally careful not to nudge or otherwise move the robot or the camera. This error is only valid for a specific calibration run, so if anything changes you will have to re-do it! However, note that it is OK to move the paper provided you take a new image and re-locate the markers each time you move the robot.

It is likely that you will be able to get the error close to zero but not exactly. This is due to a multitude of effects. In your report, you will explore why this is the case. This procedure to reduce error from measurements will be important in lab 5.

Part 2: Simple Trajectories

To keep your workload reasonable, for this part, you do not need to be concerned about whether your errors are reduced (that is, once you finish part 1 in this lab, if the robot gets bumped you should just re-calibrate it using the scripts, don't worry about re-computing the error corrections).

We are going to simulate a simple pick-and-place task. In such a task, the robot would have a set of goal positions that it must move between, and complete an interaction at each location. For instance, the first location might hold an object, which the robot must pick up (the first interaction), move to the second location, and put it down. In lab 4, we will not be interacting with real objects, and will instead rely on the pen.

Write a python script that takes in a sequence of ArUco marker IDs to move between, of arbitrary length. When the user enters a negative number, stop taking in IDs. An example might be:

```
>> 1 2 1 1 2 3 2 3 1 3 1 -1
```

The robot must move to each marker, keeping its pen off the table while moving. Then it must place a dot on the printout, reset to a higher position, and move to the next marker. If the same marker is encountered in sequence (like 1 1) the robot must place a dot, move up, and place another dot. It is not advised to continuously take new images and re-locate the ArUco markers during a trajectory motion. You will explore why in the report.

At all times the robot must remain inside of its restricted workspace. If any markers are outside of this workspace, or are not detected by the camera, the robot should ignore them and attempt to move to the next one in the list. If all markers are outside of the workspace, the robot should not move at all.

For each motion, record the position of the marker from the camera and the *pen tip's* real position when the dot is placed. Note: you may choose to first record the robot's tool frame position then compute the pen tip position. You should also read the report requirements before performing this experiment to make sure you are carefully observing what you need to in order to answer the questions there.

For testing purposes, your group will be provided with a sequence of markers to move to by the lab instructor.

Report Requirements

Part 1 (20 marks)

For part 1, describe your procedure for choosing points to move to, measuring error, and correcting it. How many experiments did you choose to run? When and how did you know that you had reduced error as much as you could?

Show your errors in a table after each correction you made. This table should clearly show a reduction in overall error by the last experiment.

Comment on the following:

1. Why was there error in the first place? Analyze at least 3 sources of error between the estimated transformation and the real one. Do this *numerically* and with a sound explanation suitable to fourth year engineering students. It is not acceptable to simply say "human error" or

“printing errors”. If you identify a source of error, quantify it using measurements, and prove that it is significant.

2. Design a new method of calibration that can reduce this error without having to run experiments and manually adjust things. You do not have to implement this, but your design should be based on your observations. It can include modifications to the robot and calibration setup, or simply additional procedures that must be followed before attempting calibration.

Part 2 (10 marks)

Present and discuss your data and observations. Was the position of the dots accurate? How do you know? What measurements did you take to determine this? What were their results?

Comment on why it was not advised to take several images as you moved the robot. What possible errors could this decision (to not take more images) have introduced?

Formatting (5 marks)

In this report, all equations must be typed using an equation editor. Figures and tables, if present, must be properly introduced in the text and properly captioned. Writing is expected to be high-quality technical writing suitable for a fourth-year engineering class. Provide succinct but sufficiently detailed explanations to guide the user through your thought process. You should be presenting numeric results and computations. Intermediate algebraic steps that are obvious may be omitted, but when in doubt add more detail rather than less.

Hand-drawn diagrams are sufficient for this lab, but they must be reasonably high quality. Use a ruler if you can't draw a straight line freehand. Scan anything done on paper with a high-resolution scanner. Cell phone pictures are not suitable. Diagrams may be drawn in software as well.

Code and/or pseudocode is not required as part of your submission for this lab. However, if you feel it would enhance your ability to explain what you did, you may include it. The code will be shown in the main body of the report. It is expected that code is properly formatted with indentation respected. For students using Notepad++, the plugin “NppExport” will help *a lot* with this. Code is *not* a substitute for mathematics – if you are going to describe a calculation, you may not just show your code instead of properly typed equations.

Report Grading

The report is out of 35 marks. The two main sections are marked according to the marking scheme below, normalized to a 100 point scale. Part marks are possible:

+100: Completely correct and comprehensive

+80: Completely correct but missing some minor points, or insufficient justifications have been given

+60: Minor factual, theoretical, or experimental errors, but the report is comprehensive

+40: Considerable factual, theoretical, or experimental errors but still broadly correct, report is comprehensive

+20: Significant factual, theoretical, or experimental errors, largely incorrect or missing substantial content regardless of correctness

0: Not submitted or completely incorrect

In addition, the 5 style marks are given for:

+1 Grammar

+1 Written in professional language

+1 Clarity

+1 Format, including correct IEEE formatting of images, tables, and references (if used in the report)

+1 Images, equations, and tables are high quality